

AgentCamp

📅 14 March 2026

🕒 09:00 AM – 04:00 PM CST

📍 Harbour Space University
Barcelona, Spain





Designing Deterministic Systems Around Non-Deterministic LLMs

📅 14 March 2026 ⌚ 09:00 AM – 04:00 PM CST

📍 Harbour Space University, Barcelona, Spain



Sepehr Samadi
Senior Frontend Engineer



encamina
PIENSA EN COLORES



it≡quia

isolutions'



Azure OpenAI: Production-Grade LLM Platform

Why enterprises choose Azure for LLM applications

Enterprise Security

Private endpoints, managed identity

Data Residency

Regional deployment for compliance

Provisioned Throughput

PTUs guarantee capacity

Content Safety

Built-in harmful content filtering

AI Search Integration

Hybrid vector + keyword search

Azure Monitor

Native observability & tracing

Today's Demo Setup

Platform: GitHub Models

- Microsoft-backed, free tier
- Same GPT-4o models
- Zero setup for demos

Production Path

Swap endpoint + auth → Azure OpenAI

Add: AI Search, Content Safety, Monitor

3 lines of code to migrate

4 Production Challenges

Each tackled with a **Basic** → **Advanced** solution arc

1

Tool Misuse

Basic → Advanced solution arc

- ⚠ Agent calls wrong tools, causes side effects
- ✅ Basic: Tool schemas with constraints
- 🚀 Advanced: Sandboxed execution + idempotency

2

Hallucination & Context Drift

- ⚠ Confident hallucination + context loss
- ✅ Basic: LLM-as-judge validation
- 🚀 Advanced: Semantic entropy + context management

3

Recursive Loops & Token Exhaustion

- ⚠ Infinite loops + budget depletion
- ✅ Basic: Iteration limits + token counting
- 🚀 Advanced: Loop detection + adaptive budgeting

4

No Observability

- ⚠ Black box execution traces
- ✅ Basic: JSON logging
- 🚀 Advanced: OpenTelemetry + orchestration



Structured Output Contracts



Ensuring predictable response formats. This is the baseline before we tackle harder problems

✗ The Problem: Unstructured LLM Response

"Sure! The ticket seems urgent, probably
priority 4 or 5, and yes I'd escalate it.
The category is billing I think?"

Structured Outputs: Three Implementation Patterns



JSON Mode

```
response =
client.chat.completions.create(
    model="gpt-4o",
    response_format={
        "type": "json_object"
    }
)
# Valid JSON, no schema
```

Use when: Simple JSON needed

Pydantic Schema

```
class Response(BaseModel):
    priority: int
    escalate: bool

response = client.beta.chat\
.completions.parse(
    model="gpt-4o",
    response_format=Response
)
```

Use when: Type safety + validation

Function Calling

```
tools = [{
    "type": "function",
    "function": {
        "name": "analyze_ticket",
        "parameters": {
            "properties": {
                "priority": {
                    "type": "integer"
                }
            }
        }
    }
}]
```

Use when: Tool calling workflows

Problem 1: Tool Misuse

Calling wrong tools in wrong sequences with irreversible side effects



What We Asked For

"Clean up all tickets from
`spam_bot@evil.com`"

Agent's Reasoning

"Clean up all tickets' → I should call
`delete_all_user_tickets()`"

★ The Disaster

47 TICKETS DELETED - IRREVERSIBLE

Root Causes

- Agent called a dangerous, irreversible tool
- No approval workflow enforced
- No sandbox. Executed directly in production
- No rollback or undo mechanism

Real-World Equivalents

- Deleted production database
- Charged wrong customer £10,000
- Sent email to 50,000 users
- Modified critical infrastructure config

Basic Solution: Tool Schemas with Constraints



Only expose safe tools, and use precise descriptions to guide agent behaviour

```
SAFE_TOOLS = [  
  {  
    "type": "function",  
    "function": {  
      "name": "read_ticket",  
      "description": "SAFE - read-only.  
        Use to inspect tickets.",  
      "parameters": {...}  
    }  
  },  
  {  
    "type": "function",  
    "function": {  
      "name": "mark_as_spam",  
      "description": "SAFE - reversible.  
        Does NOT delete.",  
      "parameters": {...}  
    }  
  }  
]  
# delete_all_user_tickets NOT included!
```

✓ Benefits

- Agent can only call exposed tools
- Clear descriptions guide selection
- Simple to implement

⚠ Limitation

Agent could still call `mark_as_spam` 1,000 times in a loop – basic schemas alone don't prevent repetition.

90%

Success Rate

1.0×

Relative Cost

☁ **Azure OpenAI** Function Calling provides robust tool schema validation out of the box.

Advanced Solution: Sandboxed Execution + Idempotency + Approval



Treat tools as unreliable distributed services — same engineering rigour as production APIs

```
class SandboxedToolExecutor:
    def execute(self, tool_name, params):
        # 1. Idempotency check
        if in_cache(tool_name, params):
            return cached_result

        # 2. Approval for dangerous tools
        if tool_name in DANGEROUS:
            if not request_approval(
                tool_name, params):
                raise PermissionError("Denied")

        # 3. Execute in sandbox
        result = run_in_sandbox(
            tool_name, params)

        # 4. Cache result
        cache[key] = result
        return result
```

Idempotency Cache

Same call twice → cached result. Safe to retry without duplicate side effects.

Approval Workflow

Slack/Teams notification with human approval gate for all critical operations.

Sandbox Isolation

Azure Container Apps with network isolation and resource limits per execution.

🔗 **Azure Durable Functions** (built-in idempotency) + **Azure Container Apps** (sandboxing)

Feature

Basic

Advanced

Prevents wrong tool



Prevents repeated calls



Approval for dangerous ops



Safe to retry



► LIVE DEMO

Cycle 1



Problem 2.1: The Model Hallucinates with Confidence



Fabricating policy citations with high certainty — the most dangerous failure mode

User Ticket

"According to Section 4.2 of your SLA, any interruption exceeding 2 hours qualifies for an immediate same-day refund with 15% service credit... Per Article 9(b) and Billing Policy 7.3.2, this requires immediate executive escalation."

Agent Response

"Based on Section 4.2 and Policy 7.3.2, you qualify for immediate processing with service credits as specified." **Confidence: 0.92**

✗ Fabricated

- Section 4.2: DOESN'T EXIST
- Article 9(b): DOESN'T EXIST
- Policy 7.3.2: DOESN'T EXIST

Reality Check

The model fabricates citations **with high confidence** — standard output validation cannot catch this.

✓ Actual Knowledge Base

- kb-001: Refunds within 5–7 business days
- kb-003: Enterprise SLA is 4-hour response

Problem 2.2: Context Drift in Multi-Turn Conversations

The agent contradicts itself as the context window fills and older turns are truncated



Turn 1 - Correct

User: "What's the refund policy?"

Agent: "Refunds processed within 5–7 business days once approved."

Turn 7 - Contradicts Turn 1

User: "Wait, what was the refund policy again?"

Agent: "Refunds are issued **immediately** upon request for Enterprise customers with same-day processing."

Basic Solution: LLM-as-Judge Validation

Use a second LLM to validate groundedness before returning a response to the user

```
class ValidationResult(BaseModel):
    is_grounded: bool
    issues: List[str]
    reasoning: str

supervisor_prompt = f"""
Does this response cite policies
not present in the knowledge base?

KB: {kb_text}
Response: {agent_response}

Rules:
1. Every claim must trace to KB
2. No invented policy numbers
3. No meaning changes via paraphrase
"""

validation = supervisor_llm.validate(prompt)
if not validation.is_grounded:
    reject_response()
```

✓ Strengths

- Catches semantic errors & paraphrasing
- No custom model required
- Simple to integrate

⚠ Limitations

- Judge itself can hallucinate
- Non-deterministic by nature
- Doesn't prevent context drift

~60-70%

Accuracy

2×

API Calls

☁ **Azure AI Content Safety** provides pre-built groundedness detection — no custom judge required.

Advanced Solution: Semantic Entropy + Context Window Management



Multi-sample consistency scoring combined with active context compression

Part 1: Semantic Entropy

```
# Generate N=5 samples
samples = [llm.generate(prompt)
            for _ in range(5)]

# Measure disagreement
categories = [s.category for s in samples]
entropy = -sum(
    p * log(p)
    for p in Counter(categories).values()
)

# High entropy = uncertain = hallucination
if entropy > THRESHOLD:
    flag_as_hallucination()
```

Part 2: Context Management

```
class ContextWindowManager:
    def get_managed_context(self):
        recent = conversation[-10:]
        older = conversation[:-10]
        summary = llm.summarize(older)
        return [system_prompt,
                summary] + recent
```

Metric	LLM Judge	Semantic Entropy
Hallucination Detection	~60-80%	~95%+
Context Drift Prevention	✗	✓
API Calls	2×	5-7×
Multi-turn Support	✗	✓

When to use: High-stakes domains (legal, medical, financial) and multi-turn conversational agents.

🔗 **Azure AI Search** semantic ranker + **Azure OpenAI** context optimisation

► LIVE DEMO

Cycle 2

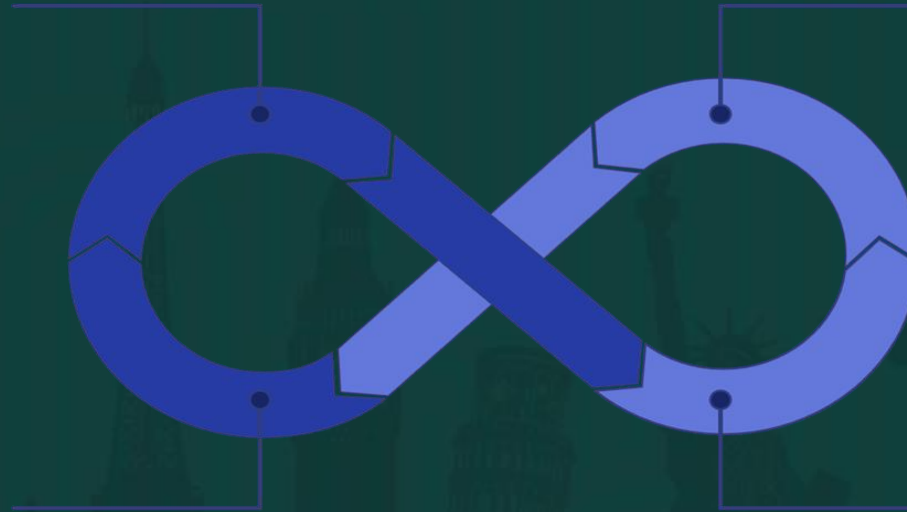


Problem 3.1: Recursive Reasoning Loops

Agent gets stuck calling the same tools repeatedly — never producing useful output

Receive Research Task

Iteration 1: Search Web



Iteration 3+: Infinite Loop

Iteration 2: Repeat Search

The Trigger

"Research the latest advancements in LLM reasoning. Keep searching until you find **definitive** information."

Why It Loops

- Agent interprets "keep searching" literally
- No loop detection or pattern recognition
- Repeated identical API calls waste budget
- Eventually hits timeout — zero useful output

Problem 3.2: Token Budget Exhaustion

Multi-step workflows consume tokens rapidly — poor planning leaves tasks incomplete

The Task: 5-Step Market Analysis

01

Search competitor data

02

Analyse pricing strategies

03

Identify market gaps

04

Generate recommendations

05

Create executive summary

Budget Maths

Each step: ~2,000 tokens

Total needed: $5 \times 2,000 = 10,000$

Budget: 8,000 tokens

SHORTFALL: 2,000 tokens

Basic Solution: Iteration Limits + Token Counting



Simple hard limits to prevent runaway execution — low complexity, adequate for linear workflows

```
class BasicWorkflowRunner:
    def __init__(
        self,
        max_iterations=10,
        max_tokens=10000):
        self.max_iterations = max_iterations
        self.max_tokens = max_tokens
        self.tokens_used = 0
        self.iterations = 0

    def execute_step(self, step, tool):
        self.iterations += 1
        if self.iterations > self.max_iterations:
            raise RuntimeError(
                "Max iterations exceeded")

        tokens_for_step = 2000
        if (self.tokens_used + tokens_for_step
            > self.max_tokens):
            raise RuntimeError(
                "Token budget exceeded")
        self.tokens_used += tokens_for_step
        return execute(tool)
```

- Prevents runaway costs
- Simple to implement
- Protects against basic loops

⚠ Limitations

- Hard limits interrupt valid workflows
- No pattern detection before limit hit
- Retry = duplicate work (not idempotent)
- No adaptive allocation

When to use: Simple linear workflows (1–3 steps), low-stakes applications.

☁ **Azure OpenAI** token usage tracking is available natively in every API response.

Advanced Solution: Loop Detection + Adaptive Token Budgeting + Idempotency



Workflows as state machines with pattern detection and dynamic resource allocation

```
class LoopDetector:
    def record_call(self, tool, params):
        if recent_calls[-4:].show_pattern():
            raise RuntimeError("Loop detected")

class AdaptiveTokenBudget:
    def can_execute_step(self, estimated):
        remaining = total - completed
        reserve = avg_tokens * remaining
        available = budget - used - reserve
        return available >= estimated

class AdvancedWorkflowExecutor:
    def execute_step(self, step, tool, params):
        # 1. Fail fast on loop pattern
        if loop_detector.is_loop(tool, params):
            raise RuntimeError("Loop detected")
        # 2. Idempotency – no duplicate work
        if in_cache(step, tool, params):
            return cached_result
        # 3. Reserve tokens for future steps
        if not budget.can_execute_step(est):
            raise RuntimeError("Budget exceeded")
        # 4. Execute and cache
        result = execute(tool, params)
        cache[key] = result
        return result
```

Loop Detection

Pattern matching on call history ($A \rightarrow B \rightarrow A \rightarrow B$). Fails fast — doesn't waste budget before stopping.

Adaptive Budgeting

Reserves tokens for remaining steps dynamically. Prevents mid-workflow failures on step 4 of 5.

Idempotency

Cached results mean retries are safe. No duplicate API calls or side effects on failure recovery.

 **Azure Durable Functions:** Built-in idempotency + checkpointing + orchestration

► LIVE DEMO

Cycle 3



Problem 4: The Agent With No Logs



Something fails. You have no idea why. Debugging takes hours of guesswork.

Your Logs

```
INFO Request received  
ERROR Error: validation failed
```

Your Code

```
logger.info("Request received")  
# ... agent runs (black box) ...  
logger.error("Error: validation failed")
```

Questions You Cannot Answer

Which ticket was being processed? What prompt was sent?

How many tokens used? What did the model return?

Which tools were called, in what order, with what parameters?

Why did validation fail? Is it safe to retry?

Basic Solution: Structured JSON Logging



Queryable logs with trace IDs and full execution context — reconstruct any timeline

```
trace = {
  "timestamp": datetime.utcnow().isoformat(),
  "trace_id": str(uuid4()),
  "agent_id": "agent-001",
  "event_type": "tool_call",
  "tool_name": "search_web",
  "parameters": {"query": "market trends"},
  "result": {"results": [...]},
  "duration_ms": 342.5,
}
logger.info(json.dumps(trace))
```

Example Queries

```
# All tool calls in a trace
jq '.[[] | select(.trace_id=="123")'
# Total duration
jq '.[[] | .duration_ms] | add'
# Failed validations
grep '"event_type":"error"' logs.json
```

✓ Benefits

- Machine-readable (jq/grep)
- Full execution context captured
- Trace ID links related events

⚠ Limitations

- Flat structure — no causal hierarchy
- Single service only
- No standard visualisation

When to use: Single-service apps, fewer than 1M requests/month.

 **Azure Monitor Logs:** Query structured logs with KQL (Kusto Query Language)

Advanced Solution: OpenTelemetry Distributed Tracing + Orchestration



Industry-standard observability with parent/child span hierarchy across services

```
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span(
    "agent.execute") as root:
    root.set_attribute("agent.task", task)
    with tracer.start_as_current_span(
        "agent.plan"):
        plan = create_plan(task)
        with tracer.start_as_current_span(
            "agent.tools"):
            for tool in plan.tools:
                with tracer.start_as_current_span(
                    f"tool.{tool}"):
                    result = execute_tool(tool)
            with tracer.start_as_current_span(
                "agent.respond"):
                return generate_response(results)
```

Span Hierarchy

```
agent.execute      (1250ms)
├─ agent.plan      ( 145ms)
├─ agent.tools      ( 980ms)
│   ├─ tool.search  ( 342ms)
│   ├─ tool.analyze ( 521ms)
│   └─ tool.summarize ( 117ms)
└─ agent.respond    ( 125ms)
```

Orchestration Patterns

- Fan-out/Fan-in: parallel research
- Sequential: multi-step analysis
- Conditional: branch on results
- Human-in-loop: approval gates

🔗 **Azure Monitor Application Insights:** Native OTel support + Application Map visualisation

► LIVE DEMO

Cycle 4



Takeaways

1 Treat LLMs as Unreliable Distributed Components

Apply SRE principles: circuit breakers, retries, observability. Build deterministic boundaries — don't rely on model behaviour alone.

2 Graduate from Basic → Advanced as You Scale

Start with schemas, LLM judge, iteration limits, JSON logs. Don't over-engineer day one — evolve when metrics demand it.

3 Every Advanced Pattern Has a Cost/Benefit Tradeoff

Semantic entropy: 5× cost → 95% accuracy. Idempotent execution: +20% latency → safe retries. Let data drive decisions.

4 Azure Makes Production Easier

Content Safety, Durable Functions, Application Insights — purpose-built for these patterns. Three lines to migrate from GitHub Models.

5 The Patterns Are Universal

Circuit breakers, semantic entropy, loop detection, OTel — industry-standard techniques that work across GPT-4, Claude, Gemini, and Llama.

6 Measure Everything, Optimise Based on Metrics

Track hallucination rate, invalid state rate, and P95 latency. Let those numbers — not intuition — drive your architecture decisions.



Q&A

AgentCamp

All notebooks and code samples from today's session:

[github.com/\[your-username\]/llm-deterministic-systems](https://github.com/[your-username]/llm-deterministic-systems)

encamina
PIENSA EN COLORES



it≡quia

isolutions'



Join the Global AI Meetup

